

Retailligence

Forecasting next week's sales for every department of every store, from sales history, store, product, promotion, month and holiday data.

421,570

sales rows

45 × 81

stores × departments

\$1,277

average miss / dept-week

0.987

test R^2 (*share of the ups and downs the model explains; 1 = perfect*)

B.Tech CSE 2024–28 · Semester V · Machine Learning Fundamentals · Mini-project report

About the project

Retaillignce is a **next-week sales forecaster**

For one department of one Walmart store, it predicts next week's sales so the manager can order the right stock and roster the right staff.

Built on **real data**

Walmart's 45 stores, 81 departments, 143 weeks (Feb 2010 – Oct 2012): 421,570 weekly sales records.

Everything is **reproducible**

One notebook runs the whole pipeline (*the same steps, in the same order, for every row*) (also live on Google Colab); the app, this report and 21 automated tests all read its outputs.

Chosen model: **Polynomial Regression** (*linear*

regression plus squared and multiplied inputs, so the line can bend) **(degree 2)**

17.9% lower error than "next week = last week" on 17 weeks it never saw.

```
Retail Sales Prediction - Group 12/
├── .gitignore
├── .streamlit/
│   └── config.toml
├── CONTRIBUTIONS.md
├── README.md    # how to run
├── Retaillignce-Presentation.pdf  # 9-slide viva deck
├── Retaillignce-Report.pdf       # full report
├── Retaillignce.ipynb           # full pipeline: EDA → model → evaluation
├── app.py                       # Streamlit app
├── data/
│   ├── features.csv            # store-week context
│   ├── stores.csv              # 45 stores
│   └── train.csv                # 421,570 sales rows
├── figures/ (16 files)          # 16 charts
├── models/
│   └── best_model.joblib        # saved model
├── problem-statement.pdf        # official brief (Group 12 + common rules)
├── report/
│   ├── build_report.py          # builds the report + presentation PDFs
│   ├── report.css
│   └── screenshots.py           # app screenshots
├── reports/
│   ├── eda.json                 # all data numbers
│   └── results.json             # all model numbers
├── requirements-dev.txt
├── requirements.txt             # pinned libraries
├── screenshots/ (5 files)       # 5 app screenshots
├── src/
│   ├── features.py              # shared feature pipeline
│   └── theme.py                 # shared colours
└── tests/                       # 21 automated tests
    ├── test_app.py
    ├── test_features.py
    └── test_model.py
```

SUMMARY

Abstract

Retail stores must decide every week how much stock to order and how many staff to roster. Both depend on a forecast of next week's sales. Using the public Walmart Recruiting – Store Sales Forecasting data (421,570 store-department-weeks), we built features *(input columns the model uses)* from sales history (last week, last 4 weeks, the same week last year and last year's jump for that week), extracted dates into month, holiday and pre-Christmas flags, and combined them with store, department and promotion information. Linear and Polynomial Regression *(linear regression plus squared and multiplied inputs, so the line can bend)* were compared against three rule-of-thumb baselines *(simple rules the model must beat)* with 5-fold time-series cross-validation *(testing on several held-back blocks of weeks)*, and the chosen model was tested once on the final 17 weeks. Polynomial Regression (degree 2) reached MAE *(average miss in dollars)* \$1,277, RMSE *(typical miss in dollars, big misses punished more)* \$2,544 and R^2 *(share of the ups and downs the model explains; 1 = perfect)* 0.987, cutting the error of the best rule of thumb by 7.9%. Holiday spikes remain the hardest case, because the data contains only one holiday season to learn from. The model is deployed as a Streamlit *(Python tool for building a web app)* app that shows the forecast, an 80% range *(8 out of 10 past forecasts landed inside it)* and a rule-of-thumb cross-check



Introduction and motivation

Forecasts drive stock and staff

Order too much and cash sits on shelves or spoils; order too little and shelves go empty on the busiest days.

Sales are not steady

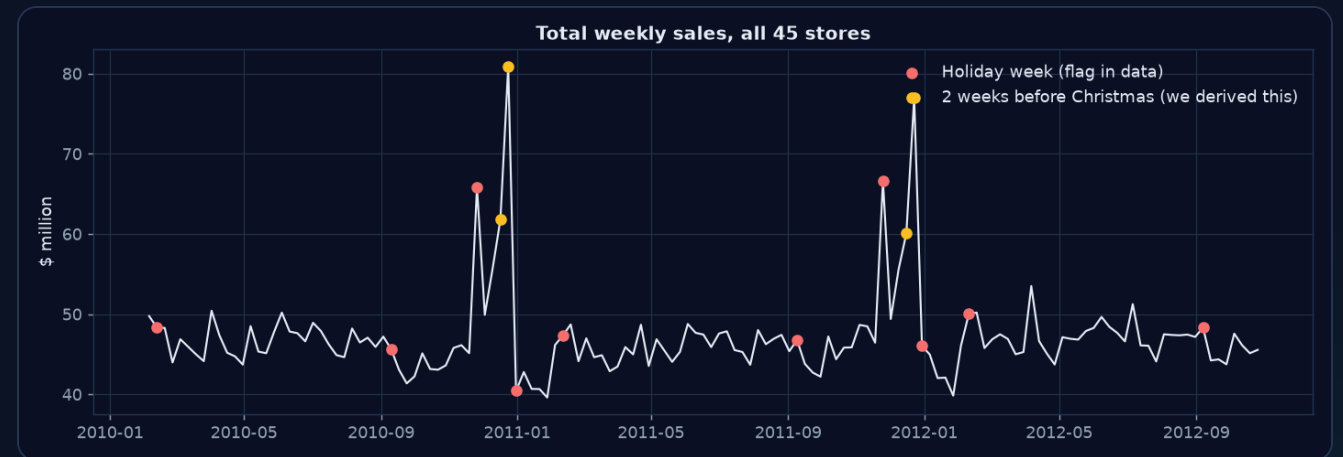
Chain-wide sales jump to \$80.9M in the week before Christmas, against $\approx$ \$46.14M in a normal week.

Every department is different

The top 10 departments make 46.0% of all sales; a forecast must work for a \$50 department and a \$500,000 one.

Goal: one model for all 3,331 store-departments

that uses only what is known before the week starts.



Problem statement and objectives

Group 12 – Retail Sales Prediction. Build a model that predicts retail sales using historical sales, product, store, promotion, month, holiday and other suitable information.

Target *(the number we predict)*

`Weekly_Sales` : dollars sold by one department of one store in one week.

Problem type

Supervised **regression** *(predicting a number, not a category)* on time-ordered data, i.e. a forecasting problem: never train on the future.

Objective

Beat the simple rules a manager already uses, measured on weeks the model never saw.

MINIMUM REQUIREMENT (BRIEF)	DONE
Use sales as the target	✓ <code>Weekly_Sales</code>
EDA <i>(exploring the data with charts)</i> and appropriate date/time feature extraction	✓ 9 charts · month, holidays, pre-Christmas, history
Preprocessing and feature engineering <i>(building new input columns from raw data)</i>	✓ cleaning, imputation <i>(filling in missing values)</i> , scaling <i>(putting all numbers on the same scale)</i> , one-hot <i>(one yes/no column per category)</i> , 5 history features <i>(input columns the model uses)</i>
Implement Linear and/or Polynomial Regression <i>(linear regression plus squared and multiplied inputs, so the line can bend)</i>	✓ both, degree chosen by validation
Evaluate using MAE <i>(average miss in dollars)</i> , MSE <i>(average of the squared misses)</i> , RMSE <i>(typical miss in dollars, big misses punished more)</i> and R^2 <i>(share of the ups and downs the model explains; 1 = perfect)</i>	✓ plus WMAE <i>(MAE with holiday weeks counted 5×)</i> (Kaggle <i>(data-science competition website)</i> 's own metric)
Deploy the prediction model using Streamlit <i>(Python tool for building a web app)</i>	✓ <code>app.py</code> , 11 app tests
Final title created by the group	✓ Retailligence

Literature survey

WORK	KEY FINDING WE USED
Fildes, Ma & Kolassa (2022), <i>Retail forecasting: Research and practice</i> , IJF 38(4)	Retail demand is driven by seasonality, holidays and promotions; simple benchmarks are hard to beat and must be reported.
Makridakis, Spiliotis & Assimakopoulos (2022), <i>M5 accuracy competition</i> , IJF 38(4)	On Walmart sales, models that pool many series with calendar and history features (<i>input columns the model uses</i>) win: our one-model-for-all design.
Bojer & Meldgaard (2021), <i>Kaggle</i> (<i>data-science competition website</i>) <i>forecasting competitions</i> , IJF 37(2)	Reviews Kaggle retail competitions incl. Walmart Store Sales: lag features and holiday handling matter most.
Bergmeir & Benítez (2012), <i>Cross-validation for time series</i> , Information Sciences 191	Time-ordered evaluation (train on the past, test on the future) avoids optimistic scores: our expanding-window (<i>each check trains on all earlier weeks</i>) CV (<i>cross-validation: testing on held-back blocks of weeks</i>).
Hyndman & Athanasopoulos (2021), <i>Forecasting: Principles and Practice</i> , 3rd ed.	Naive and seasonal-naive forecasts are the standard benchmarks: our three baselines (<i>simple rules the model must beat</i>).

Existing system and its limitations

Today: rules of thumb

Managers order like last week, or like the same week last year, often in a spreadsheet.

They ignore context

"Last week" misses holidays completely; "last year" misses this year's growth or decline.

They cannot be measured

Nobody tracks how wrong the guess was, so nobody improves it.

So we measured them

All three rules are baselines (*simple rules the model must beat*) in our evaluation, on exactly the same weeks as the models.

RULE OF THUMB	TEST MAE	VS OUR MODEL
Naive: last week's sales	\$1,555	17.9% worse
Seasonal naive: same week last year	\$1,683	24.1% worse
Seasonal rule: last 4 weeks + last year's jump	\$1,386	7.9% worse
Polynomial Regression (<i>linear regression plus squared and multiplied inputs, so the line can bend</i>) (degree 2)	\$1,277	—

WORKFLOW

Proposed system and workflow

One pipeline *(the same steps, in the same order, for every row)*, **end to end**

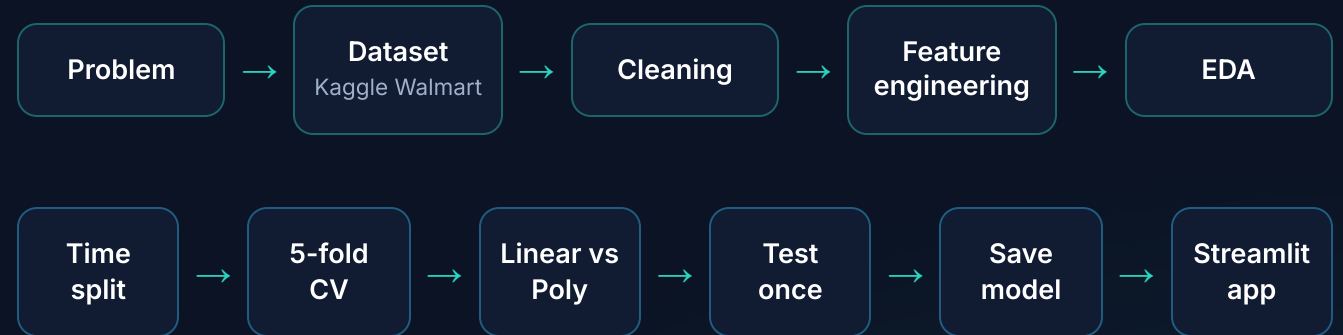
Problem → dataset → understanding → cleaning → preprocessing
→ EDA *(exploring the data with charts)* → feature engineering *(building new input columns from raw data)* → split → models → evaluation → comparison → best model → saving → Streamlit *(Python tool for building a web app)* → testing.

One shared feature module

`src/features.py` builds the inputs for the notebook and the app, so the deployed model sees exactly what it was trained on.

Honest by design

Baselines, time-ordered validation and a locked test set are part of the pipeline, not an afterthought.



Dataset description

Source

Walmart Recruiting – Store Sales Forecasting (Kaggle *(data-science competition website)*, 2014), via the Kaggle mirror `aslanahmedov/walmart-sales-forecast` (identical files, checked row counts).

Three files joined

`train.csv` (sales) + `features.csv` (store-week context) + `stores.csv` (store type and size), joined on store and date.

Target *(the number we predict)*

`Weekly_Sales` in US dollars, one row per store × department × week (weeks end on Friday).

421,570 sales rows (train.csv)	8,190 store-week rows (features <i>(input columns the model uses)</i> .csv)	45 stores (3 types)
81 departments (products)	3,331 store-department series	143 weeks · 2010-02-05 → 2012-10-26
10 holiday weeks flagged	\$6.74B total sales in the data	0 duplicate rows

Data classification: every column

COLUMN	DATA TYPE	MEANING	HOW THE MODEL USES IT	MISSING %
Store	Categorical (ID)	Store number 1–45	to look up history; not a model input (tested, removed)	0.0
Dept	Categorical (nominal)	Department = product group	one-hot <i>(one yes/no column per category)</i> encoded input	0.0
Date	Date/time	Week ending Friday	source of month, holiday and history features <i>(input columns the model uses)</i>	0.0
Weekly_Sales	Numerical, continuous	Dollars sold that week	TARGET; its past values become the history inputs	0.0
IsHoliday	Binary	Super Bowl, Labor Day, Thanksgiving, Christmas	split into 4 holiday flags	0.0
Type	Categorical (nominal)	Store format A / B / C	one-hot encoded input	0.0
Size	Numerical, discrete	Store floor area (sq ft)	scaled input	0.0
Markdown1-5	Numerical, continuous	Anonymised promotional markdowns <i>(price cuts / promotions)</i> (\$)	summed into one promotion input + flag	67.5
Temperature	Numerical, continuous	Local temperature (°F)	tested, removed (no gain)	0.0
Fuel_Price	Numerical, continuous	Local fuel price (\$/gallon)	tested, removed (no gain)	0.0
CPI	Numerical, continuous	Consumer price index	tested, removed (no gain)	0.0
Unemployment	Numerical, continuous	Unemployment rate (%)	tested, removed (no gain)	0.0

Data understanding

421,570 rows × 16 columns after joining

Store, department and date identify each row; the rest are numbers, categories or flags (previous slide).

Target (the number we predict) is right-skewed

Median \$7,612, mean \$15,982, max \$693,099 per department-week.

Context columns are complete

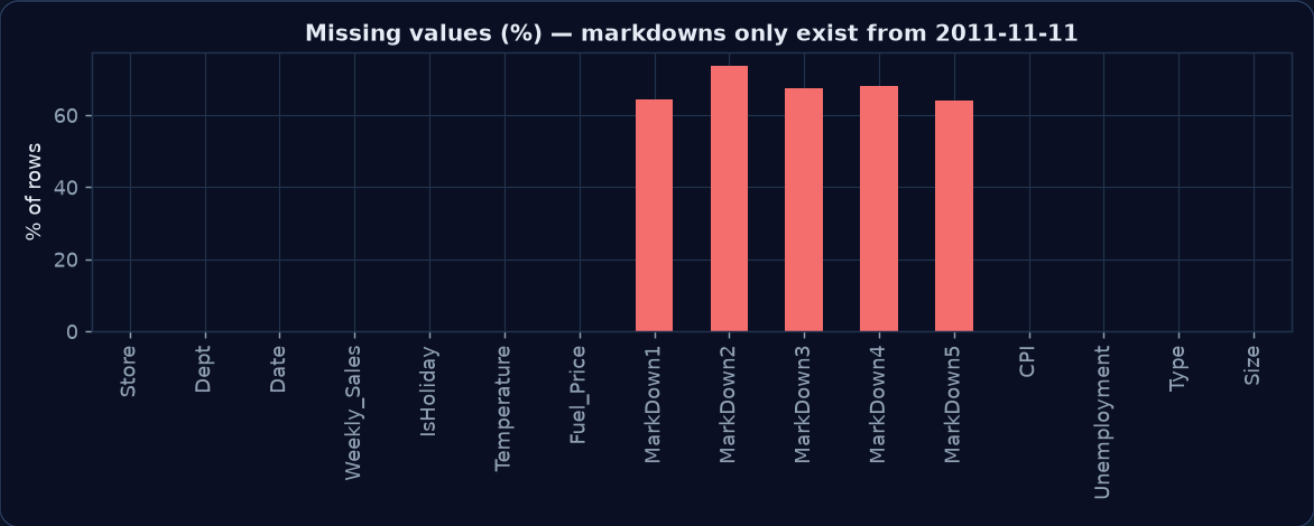
Temperature, fuel price, CPI (consumer price index: how expensive things are) and unemployment have no gaps; only markdowns (price cuts / promotions) do.

```
RangeIndex: 421570 entries, 0 to 421569
Data columns (total 16 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Store                421570 non-null int64
1   Dept                 421570 non-null int64
2   Date                 421570 non-null datetime64[us]
3   Weekly_Sales         421570 non-null float64
4   IsHoliday            421570 non-null bool
5   Temperature          421570 non-null float64
6   Fuel_Price           421570 non-null float64
7   Markdown1            150681 non-null float64
8   Markdown2            111248 non-null float64
9   Markdown3            137091 non-null float64
10  Markdown4            134967 non-null float64
11  Markdown5            151432 non-null float64
12  CPI                  421570 non-null float64
13  Unemployment          421570 non-null float64
14  Type                 421570 non-null str
15  Size                 421570 non-null int64
dtypes: bool(1), datetime64[us](1), float64(10), int64(3), str(1)
memory usage: 49.1 MB
```

	MEAN	STD	MIN	50%	MAX
Weekly_Sales	15,981.3	22,711.2	-4,988.9	7,612.0	693,099.4
Size	136,727.9	60,980.6	34,875.0	140,167.0	219,622.0
Temperature	60.1	18.4	-2.1	62.1	100.1
Fuel_Price	3.4	0.5	2.5	3.5	4.5
CPI	171.2	39.2	126.1	182.3	227.2
Unemployment	8.0	1.9	3.9	7.9	14.3

Data quality and cleaning decisions

FOUND	DECISION	WHY
1,285 rows with negative sales (net returns)	set to 0	returns are not demand
Markdowns (<i>price cuts / promotions</i>) missing 64–74% (none before 2011-11-11)	missing → 0 + <code>markdown_reported</code> flag	tell "no markdown" from "not reported"
40,483 store-dept-weeks with no row	0 sales for history; never a target (<i>the number we predict</i>)	no record = nothing sold
Holiday spikes up to \$693K	kept	real, and what we must predict
Duplicates	0 found	—



Train-test split and preprocessing pipeline

Leakage-free *(no future information sneaks into training)*

Every step is fitted on training weeks only, inside one scikit-learn `Pipeline`, then applied unchanged to validation and test weeks, and in the app.

Numeric inputs

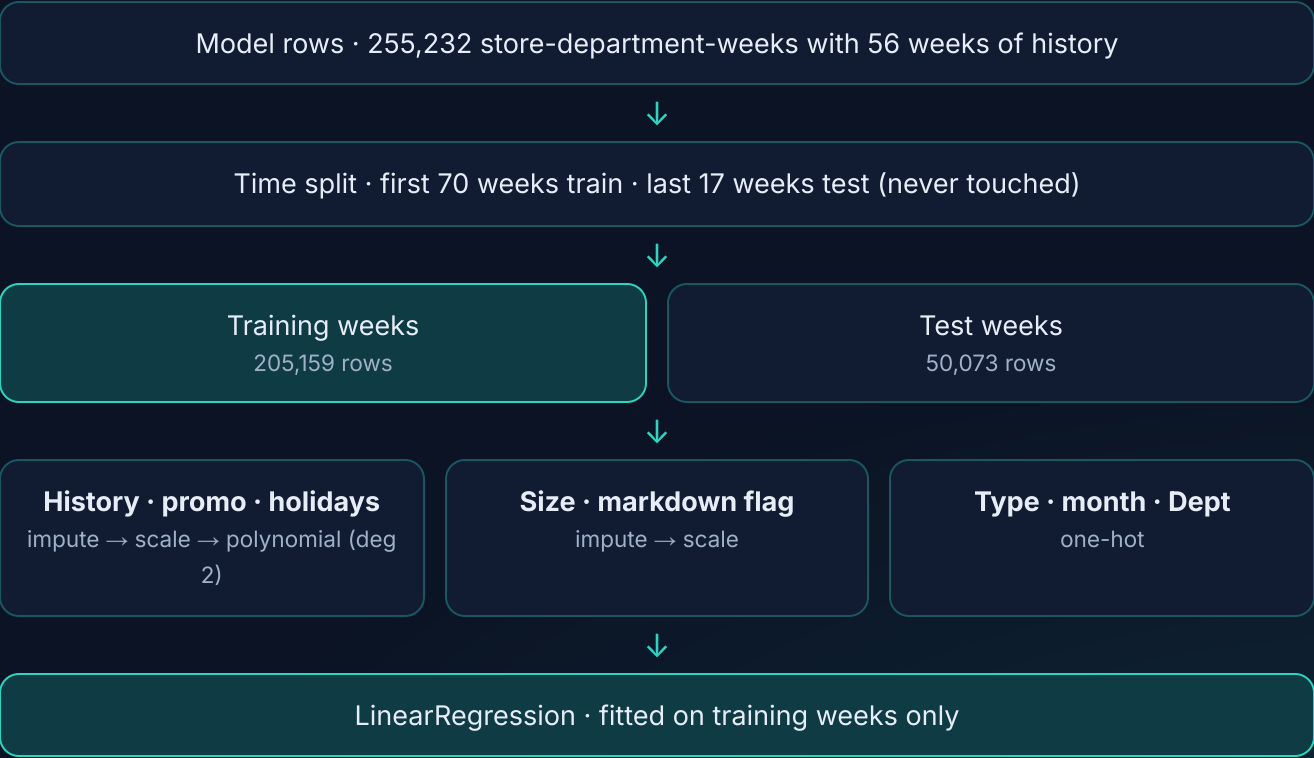
Median imputation *(filling in missing values)* → standard scaling *(putting all numbers on the same scale)* (mean 0, std 1) → polynomial terms (degree 2) for the history, promotion and holiday inputs.

Categorical inputs

One-hot encoding of store type, month and department; unseen categories are ignored safely.

Predictions

Clipped at \$0: a department cannot sell less than nothing.



Feature engineering: history, date/time, holidays

History is the strongest clue

Correlation *(how strongly two things move together, -1 to 1)* with this week's sales: last week 0.959, same week last year 0.984.

Last year's jump carries the holiday

How far this week rose above the 4 weeks before it, last year: it tells the model a spike is coming before the week happens.

Date/time extraction

From the date: month, the 4 holiday flags and the 2 pre-Christmas weeks, which are the biggest weeks of the year but not flagged in the data.

FEATURE	MEANING	FROM
sales_lag_1 , sales_lag_2	sales 1 and 2 weeks ago	Weekly_Sales
sales_rol14	average of the last 4 weeks	Weekly_Sales
sales_lag_52	same week last year	Weekly_Sales
sales_yoy_jump	last year: week 52 back – avg of weeks 53–56 back	Weekly_Sales
hol_* (4)	Super Bowl, Labor Day, Thanksgiving, Christmas	IsHoliday + month
pre_christmas	the 14 days before 25 Dec	Date
month	month of the year	Date
promo_markdown , markdown_reported	total markdown <i>(price cut / promotion)</i> , and whether reported	MarkDown1–5
Type , Size , Dept	store format, size, product group	stores / train

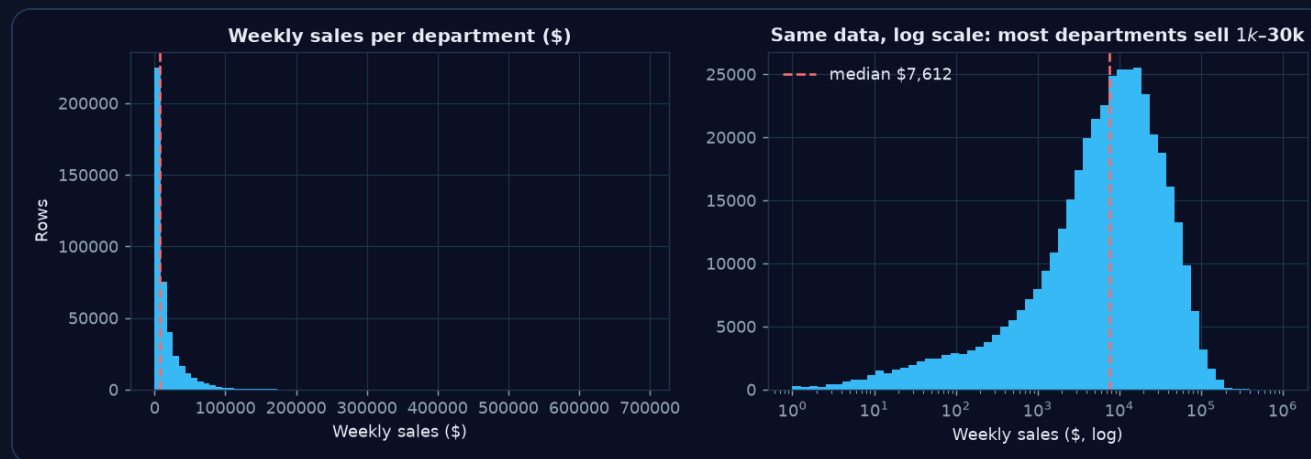
EDA 1 • The target

Sales are right-skewed

Median \$7,612 but a long tail up to \$693,099.

So errors are in dollars

Big departments dominate MAE (*average miss in dollars*) and RMSE (*typical miss in dollars, big misses punished more*); that is where the money is.



EDA 2 · Holidays multiply sales

Thanksgiving week $\approx 1.43\times$ a normal week

The 2 weeks before Christmas $\approx 1.52\times$

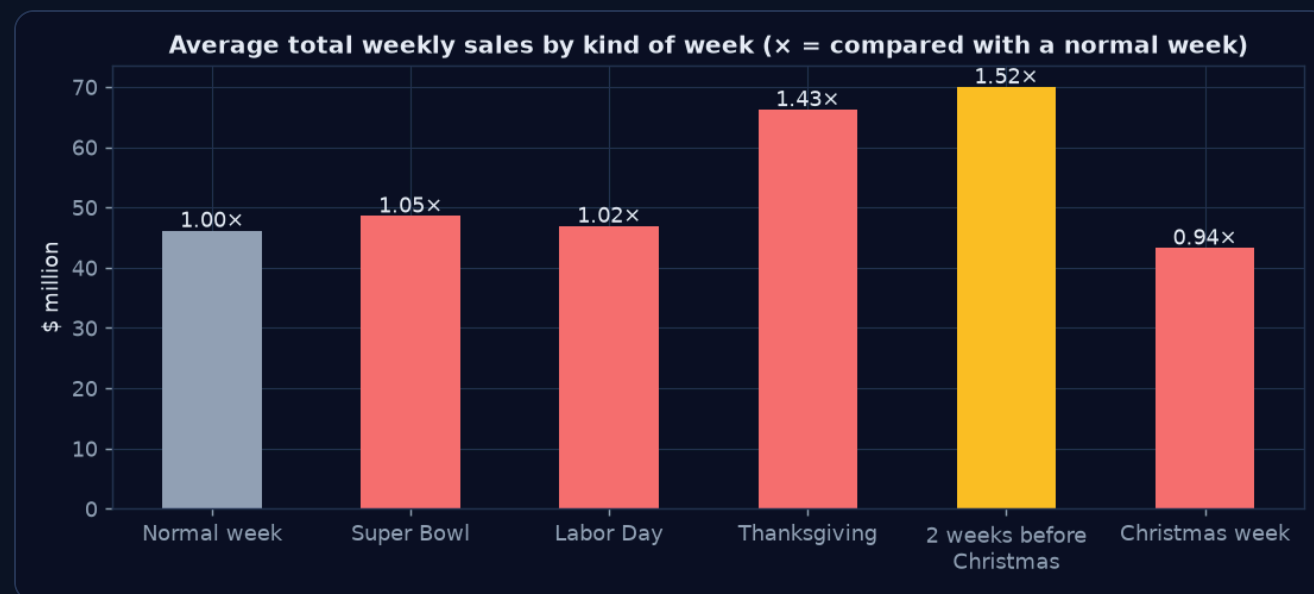
not flagged in the data, so we derived them from the date.

Christmas week itself **dips**

$0.94\times$ a normal week.

A **multiplier**, not a fixed add-on

which is why polynomial interaction terms (history $\times$ holiday) fit the problem.



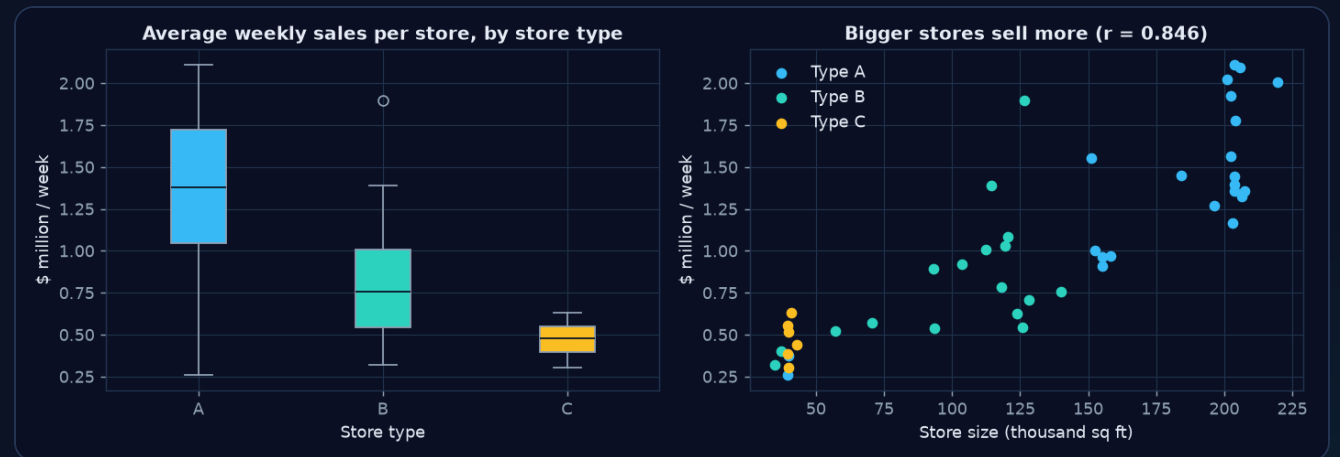
EDA 3 · Store type and size

Type A stores sell **most**

A $\approx$ \$1.38M, B $\approx$ \$0.82M, C $\approx$ \$0.47M per week.

Size predicts sales

$r =$ (correlation, -1 to 1) 0.846 between store size and store sales.



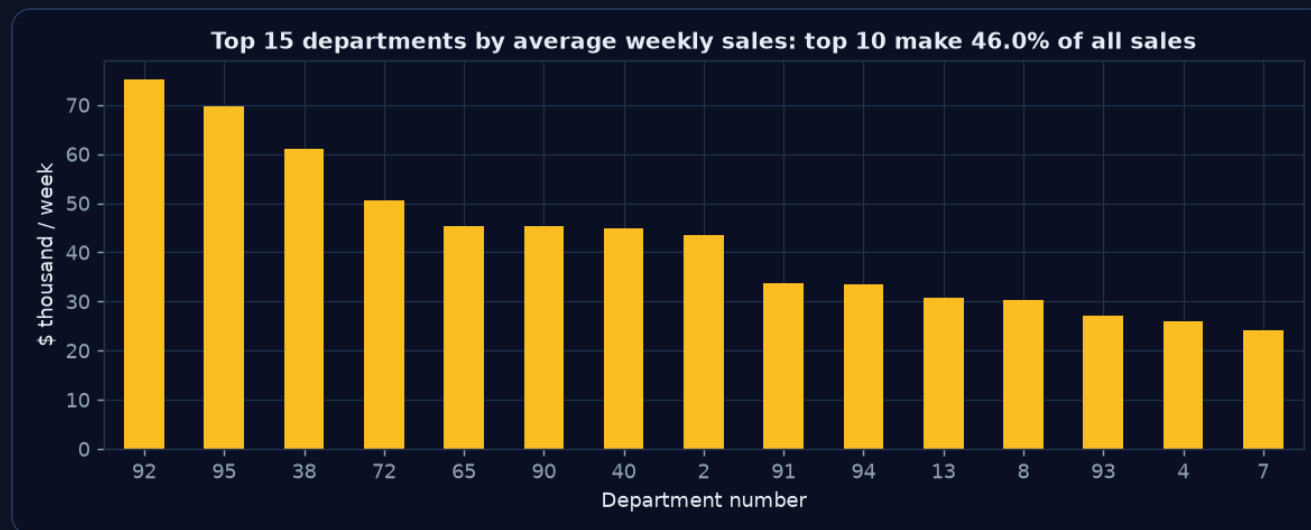
EDA 4 · Departments (the product)

A few departments carry the chain

The top 10 make 46.0% of all sales.

So the department is an input

one-hot (*one yes/no column per category*) encoded, one column per department.



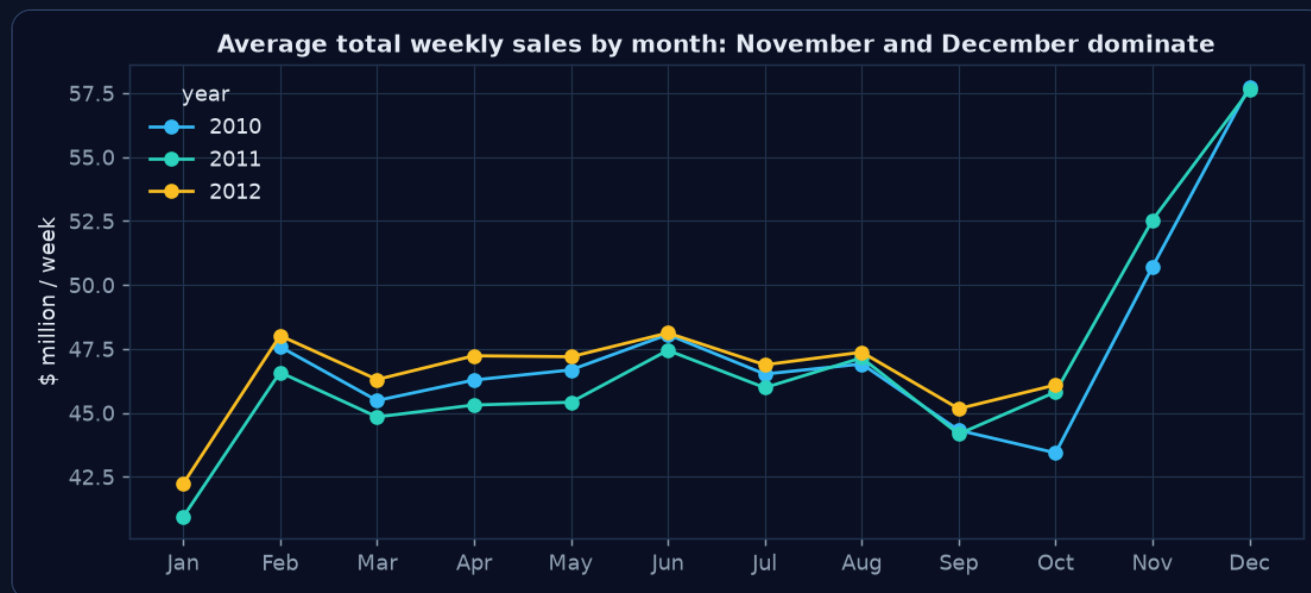
EDA 5 · Month and seasonality

November and December dominate

and the pattern repeats in both years.

Repeating pattern → "same week last year"

becomes one of the strongest features (*input columns the model uses*).



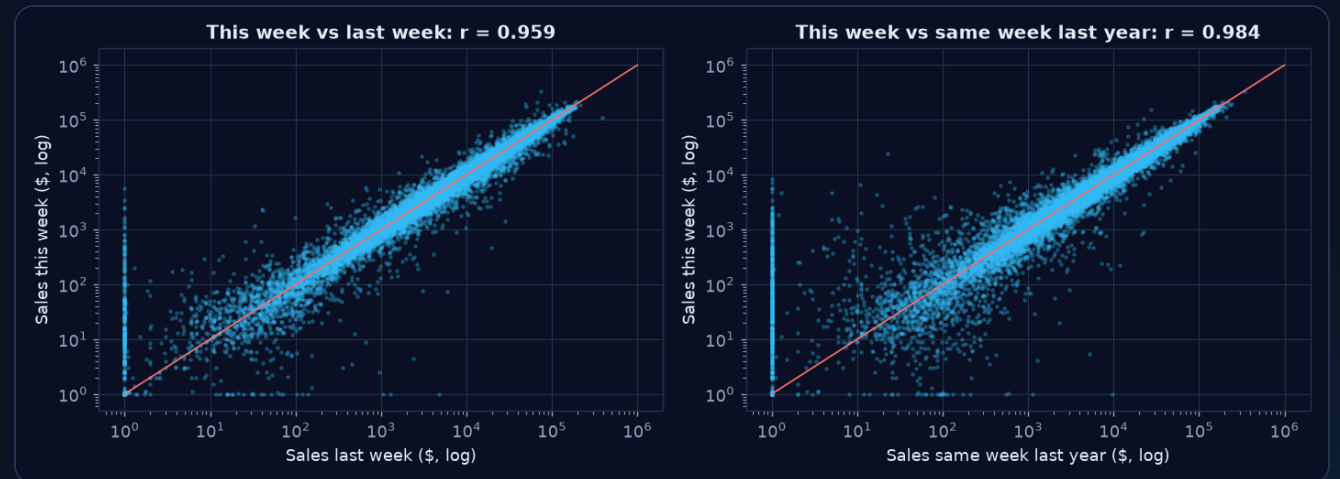
EDA 6 · History vs this week

Points hug the diagonal

r = (correlation, -1 to 1) 0.959 with last week and 0.984 with the same week last year.

History features (input columns the model uses) lead the model

confirmed later by permutation importance (how much worse the model gets when one input is scrambled).



EDA 7 · Do promotions work?

Looks like yes

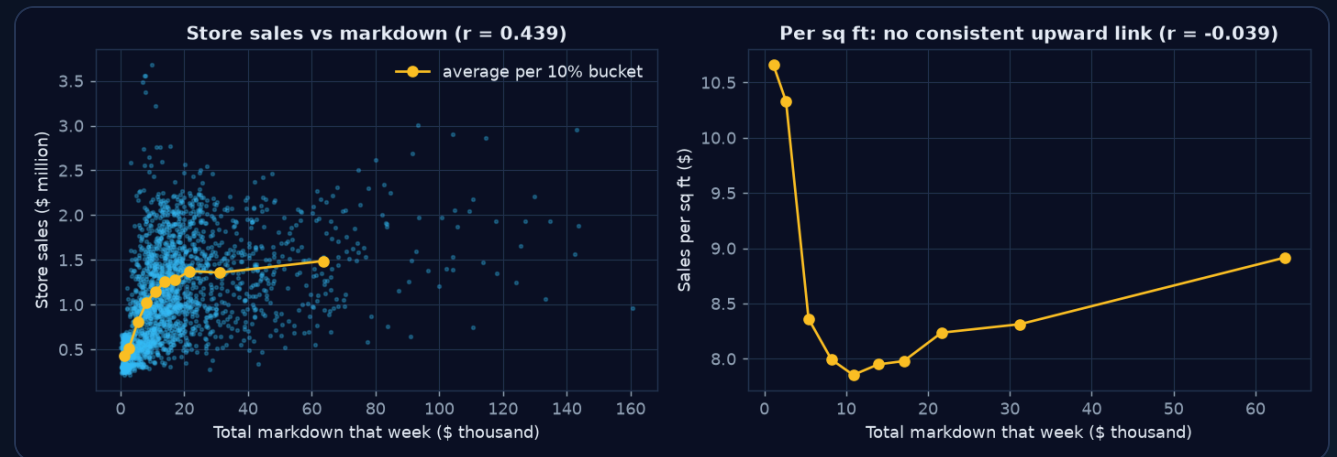
$r =$ (correlation, -1 to 1) 0.439 between markdown (price cut / promotion) and store sales...

...but it is store size

per square foot $r = -0.039$: big stores simply run bigger markdowns.

Expect markdowns to add little

once history is known; the model agrees (importance ≈ 0).



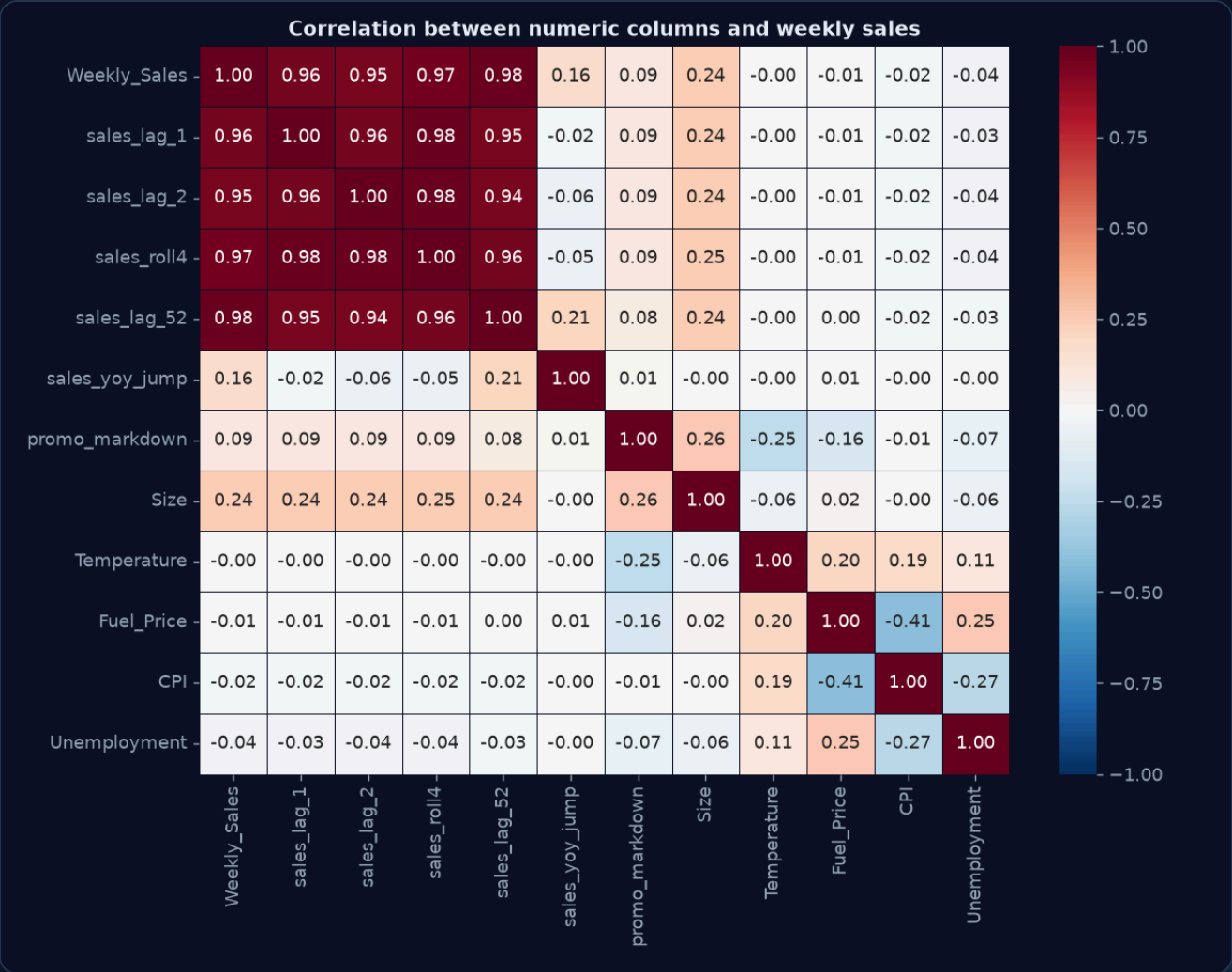
EDA 8 · Correlations

History columns $\approx 0.95+$

with this week's sales.

Economic data ≈ 0

temperature, fuel price, CPI (consumer price index: how expensive things are), unemployment; tested in feature selection, then removed.



Validation design: never peek at the future

Locked test set

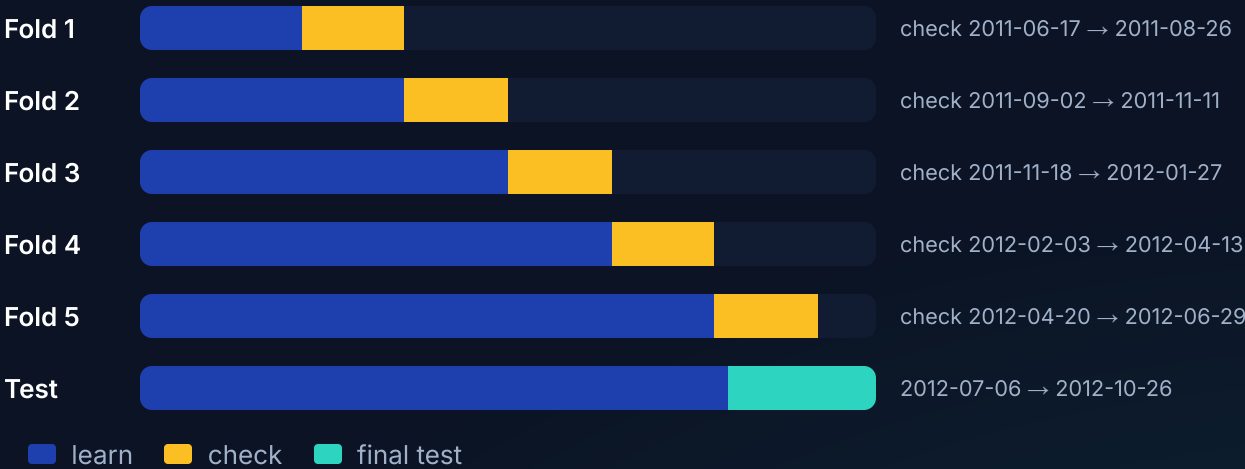
The last 17 weeks (2012-07-06 → 2012-10-26) were opened once, after the model was chosen.

Expanding-window CV *(cross-validation: testing on held-back blocks of weeks)*

5 folds inside the training weeks: always learn from earlier weeks, check on the next block.

Why not a random split?

It would let the model learn from next week to predict this week: an optimistic, useless score.



Model development

Three baselines (*simple rules the model must beat*)

Naive (last week), seasonal naive (same week last year), seasonal rule (last 4 weeks + last year's jump).

Linear Regression (*a weighted sum of the inputs*)

A weighted sum of the scaled inputs.

Polynomial Regression (*linear regression plus squared and multiplied inputs, so the line can bend*)
(degree 2)

Adds squares and pairwise products of the history, promotion and holiday inputs, e.g. last week's sales × Thanksgiving, so a holiday can multiply a department's sales.

Selection rule fixed in advance

Lowest cross-validated RMSE (*typical miss in dollars, big misses punished more*) between the two regression (*predicting a number, not a category*) models.

```
def make_model(degree, linear=LINEAR, categorical=CATEGORICAL):
    """degree 1 = Linear Regression; degree 2+ = Polynomial Regression on the history/promo/holiday columns."""
    steps = [("impute", SimpleImputer(strategy="median")), ("scale", StandardScaler())]
    if degree > 1:
        steps.append(("poly", PolynomialFeatures(degree, include_bias=False)))
    prep = ColumnTransformer([
        ("poly", Pipeline(steps), POLY),
        ("num", Pipeline([("impute", SimpleImputer(strategy="median")), ("scale", StandardScaler())]), linear),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
    ])
    return Pipeline([("prep", prep), ("model", LinearRegression())])
```

Feature selection and polynomial degree

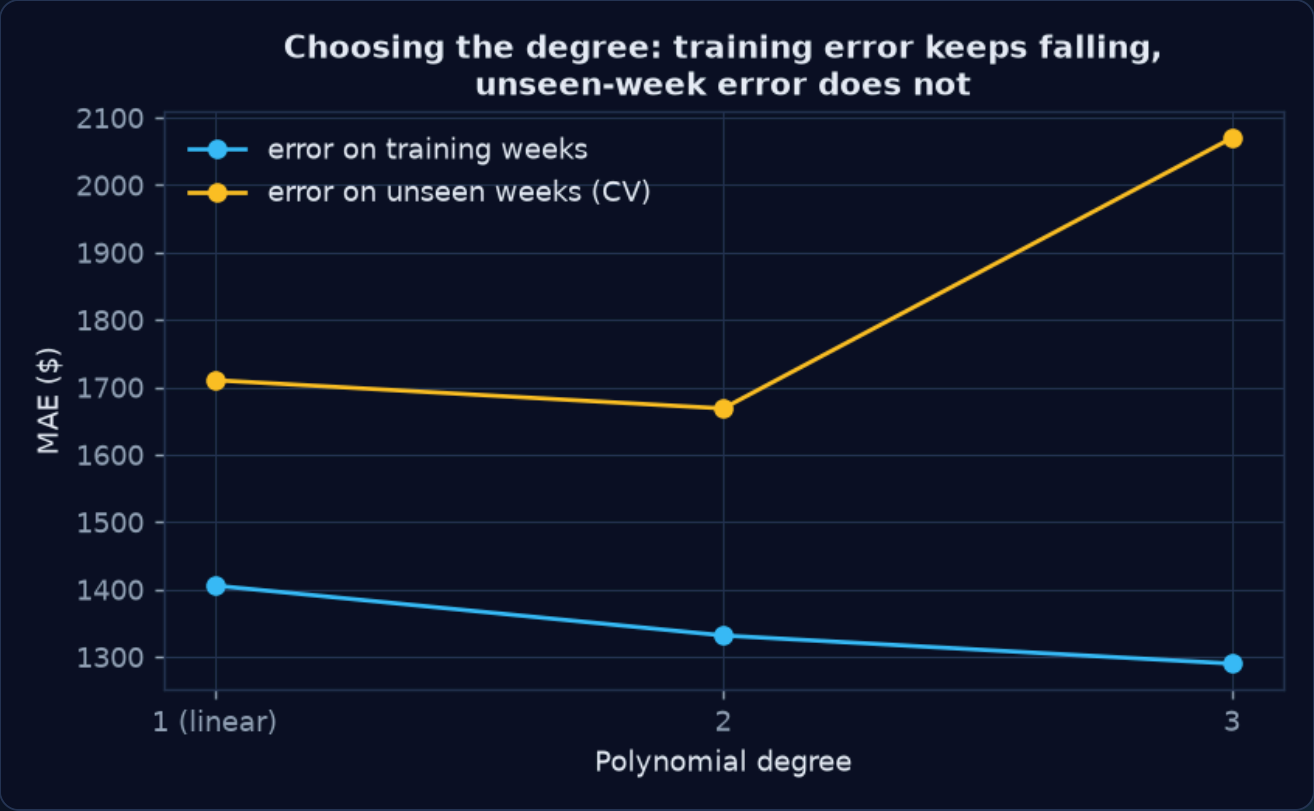
FEATURE SET (POLYNOMIAL, DEGREE 2)	CV MAE
All columns (+ store ID + economic data)	\$1,691
Without store ID	\$1,681
Without store ID and economic data (chosen)	\$1,668

Store ID and economic data removed

they did not lower cross-validated error.

Degree 2 is the sweet spot

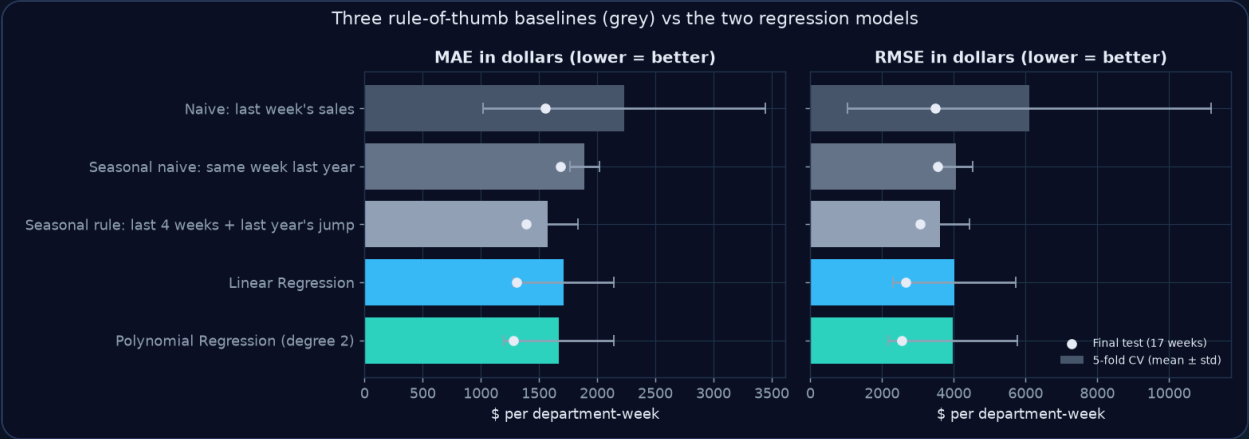
CV (cross-validation: testing on held-back blocks of weeks) MAE (average miss in dollars): degree 1 \$1,710, degree 2 \$1,668, degree 3 \$2,070. Degree 3 fits training weeks better but over-fits (memorises training weeks, does worse on new weeks).



Model evaluation and comparison

METHOD	CV MAE	TEST MAE	TEST RMSE	TEST MSE	TEST R ²	ERROR CUT VS NAIVE
Naive: last week's sales	\$2,232	\$1,555	\$3,490	12.2M	0.975	+0.0%
Seasonal naive: same week last year	\$1,893	\$1,683	\$3,550	12.6M	0.974	-8.2%
Seasonal rule: last 4 weeks + last year's jump	\$1,575	\$1,386	\$3,062	9.4M	0.981	+10.9%
Linear Regression <i>(a weighted sum of the inputs)</i>	\$1,710	\$1,309	\$2,655	7.1M	0.985	+15.8%
Polynomial Regression <i>(linear regression plus squared and multiplied inputs, so the line can bend)</i> (degree 2)	\$1,668	\$1,277	\$2,544	6.5M	0.987	+17.9%

Key: MAE = average miss in dollars · RMSE = typical miss, big misses punished more · MSE = average of the squared misses · R² = share of the ups and downs explained (1 = perfect) · CV = cross-validation, testing on held-back blocks of weeks · naive = next week equals last week



Reading the results honestly

The seasonal rule **wins cross-validation** (testing on several held-back blocks of weeks) **on average**

CV MAE (average miss in dollars) \$1,575 vs \$1,668, almost all from fold 3 (Nov 2011 – Jan 2012).

Why: no holiday season to learn from

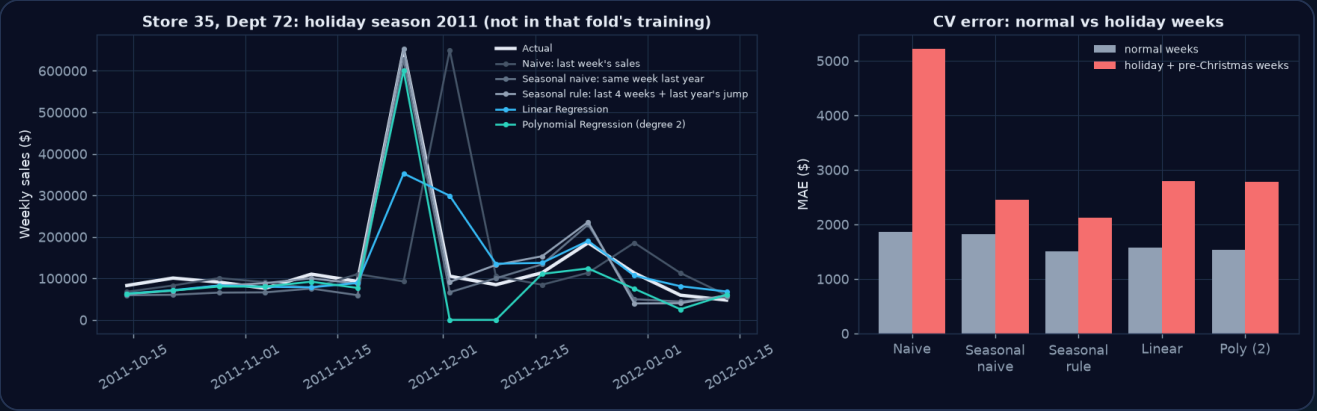
In fold 3 the models had never seen Thanksgiving or Christmas; the rule has "holiday jumps repeat" built in. Holiday-week CV MAE: rule \$2,120, polynomial \$2,785.

Once a season is in training, the model leads

Folds 4–5: polynomial \$1,592 vs rule \$1,645; final test: \$1,277 vs \$1,386.

So the app shows both

the model's forecast and the rule, with a warning on big holiday weeks.



Residual analysis

Centred near zero

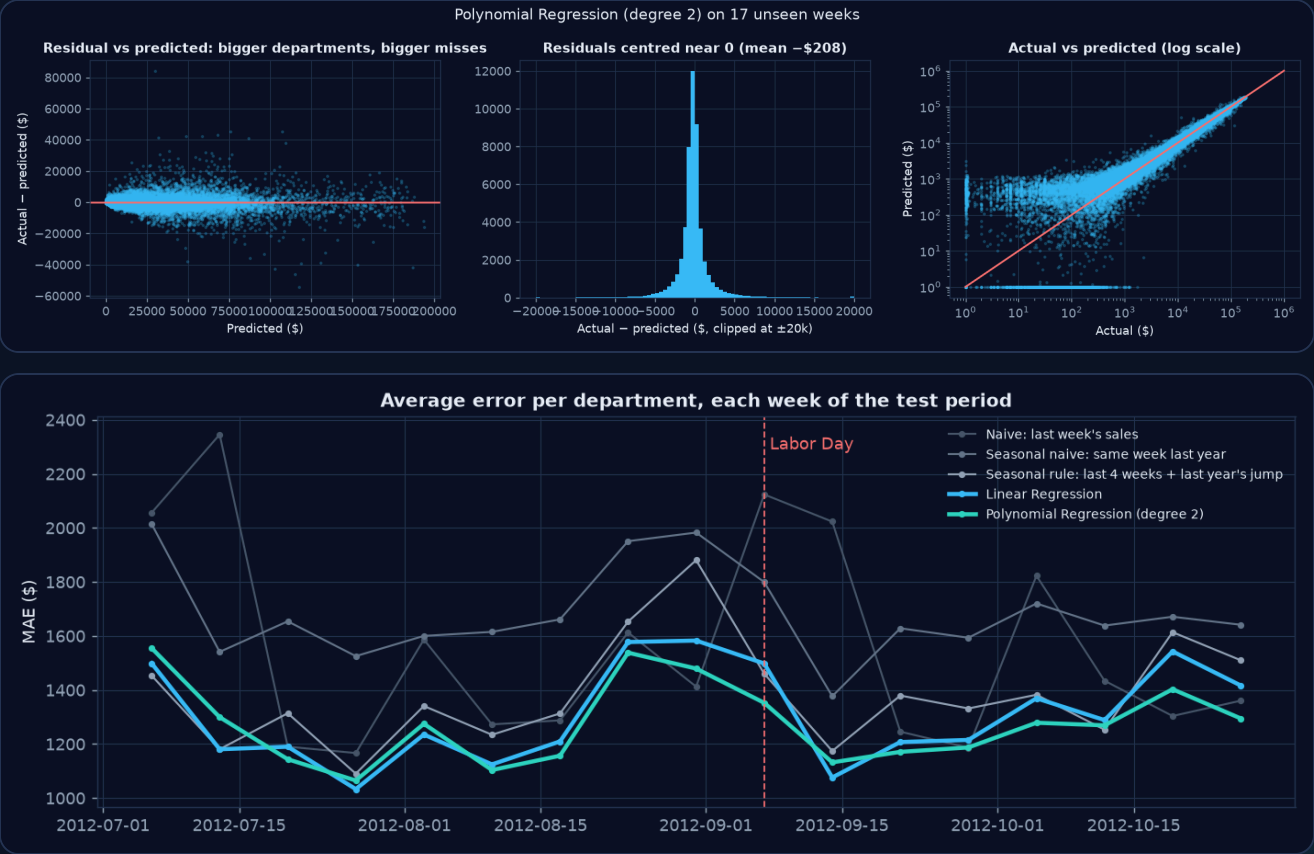
Mean residual (*actual minus predicted*) $-\$208$ (a slight over-forecast), half of all misses under $\$628$.

Misses grow with size

80% of misses under $\$1,712$; the app's range is therefore a percentage of the forecast.

Lowest average error

Of all five methods over the 17 test weeks (chart below).



RESULTS

Which inputs matter?

Recent history carries the forecast

Shuffling the 4-week average raises MAE (*average miss in dollars*) by \$7,753, the same week last year by \$5,332, last week by \$3,733.

Department and month add a little

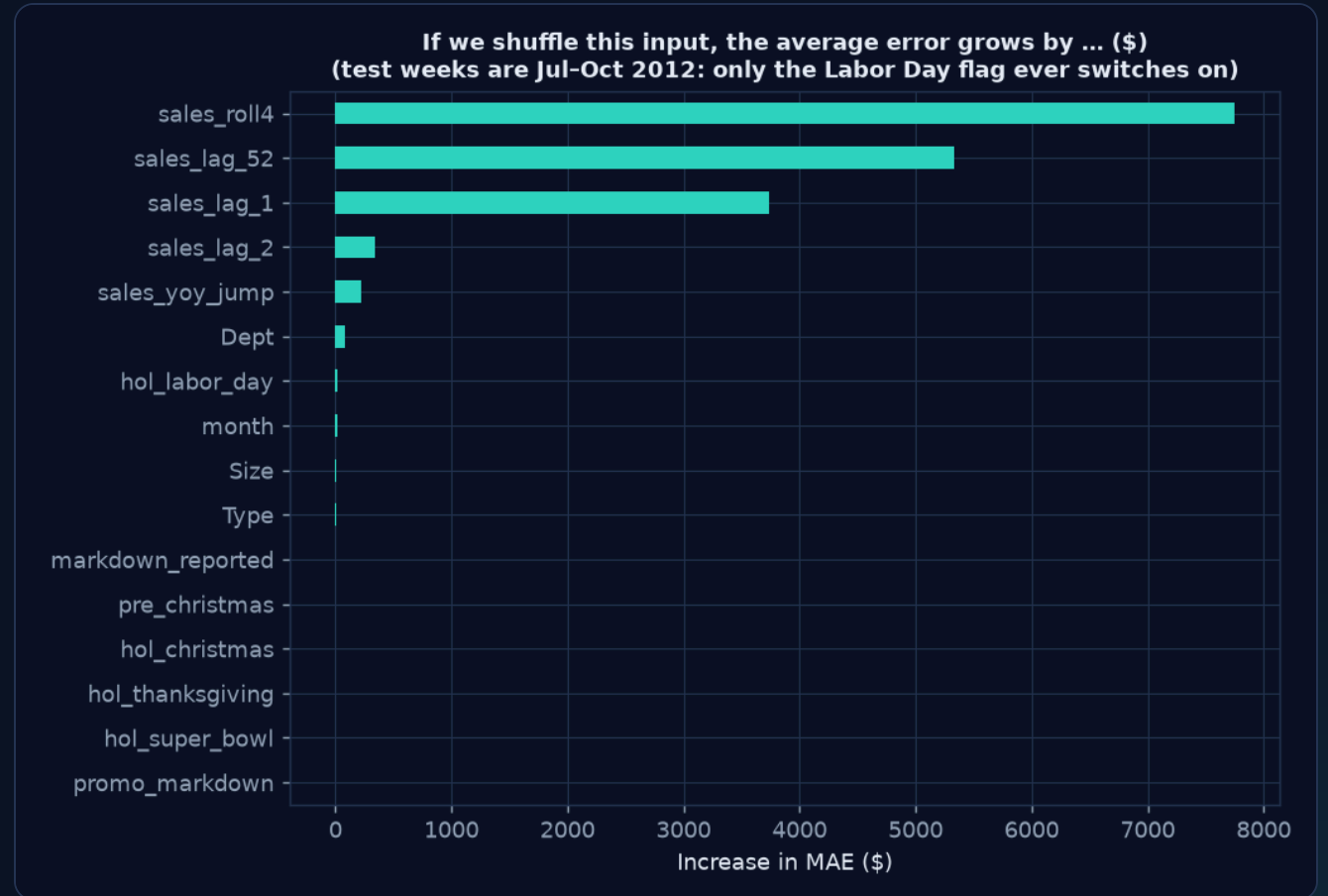
\$78 and \$21.

Markdowns (*price cuts / promotions*) add nothing measurable

-\$6, matching the EDA (*exploring the data with charts*).

Caveat

The test weeks (Jul–Oct 2012) contain no Thanksgiving or Christmas, so those flags cannot show importance here.



Streamlit deployment

Inputs (sidebar)

Store, department and week; what-if markdown (*price cut / promotion*), kind of week and last week's sales, limited to the range seen in training.

Outputs

Forecast with an 80% range (*8 out of 10 past forecasts landed inside it*), the rule-of-thumb cross-check, the actual sales for past weeks, a 16-week chart, a store snapshot and the model's test scores.

Honesty built in

Training weeks are labelled "not a fair test"; holiday weeks carry a warning.

Run it

`streamlit run app.py` , or deploy the same file on Streamlit (*Python tool for building a web app*) Community Cloud.



App: holidays, what-if and input validation

Forecast inputs

Store

Store 35 - Type B - 103,681 sq ft

Department (product group)

Dept 72

Week to forecast (week ending Friday)

2011-11-25 - training week

What-if

Total markdown (promotion) that week, \$

66304.31

Kind of week

Thanksgiving week

Last week's sales, \$

92764.63

Change any value to see how the forecast reacts. Picking a new store, department or week resets them.

ML Fundamentals Mini Project

Group 12

Polynomial Regression (degree 2)

Retaillence — Next-Week Sales Forecasting Engine

Forecast next week's sales for any department of any of 45 Walmart stores from its sales history, holidays and promotions, to plan stock and staff a week ahead.

Store 35 - Dept 72 - week of 2011-11-25

Forecast weekly sales

\$545,300

80% range: 492,406 - 598,194

Rule-of-thumb check

\$653,695

Actual sales that week

\$649,770

model under by \$38,470

This week was part of the model's training data, so it is not a fair test. Pick a week marked 'unseen test week' or 'next week' for an honest check.

The model learned these big holiday effects from a single season (2011). In testing, the rule-of-thumb was more reliable on holiday weeks — compare both numbers above.

The 16 weeks before, the same weeks last year, and this week's numbers

Weekly sales (\$)

Aug 07 - Nov 20

Actual sales Same week last year Model forecast Rule-of-thumb Actual (this week)

Store & department snapshot

Store format / size

Type B - 103,681 sq ft

Average week, past year

\$87,589

Last 4 weeks' average

\$92,529

Same week last year

\$627,963

Last year's jump for this week

+\$561,367

Kind of week

Thanksgiving week

Markdown (promotion) this week

\$66,304

Model performance (17 unseen test weeks)

Algorithm

Polynomial Regression (degree 2)

Trained on

2011-03-04 - 2013-06-29

Test R²

0.987

Test MAE / RMSE

\$1,277 / \$2,544

Error cut vs last week's sales

17.9%

Error cut vs rule-of-thumb

7.9%

Group attribution

Group 12 - ML Fundamentals

Inputs the model used for this forecast

Forecast inputs

Store

Store 1 - Type A - 151,315 sq ft

Department (product group)

Dept 1

Week to forecast (week ending Friday)

2012-10-26 - unseen test week

What-if

Total markdown (promotion) that week, \$

50000.00

Kind of week

Normal week

Last week's sales, \$

40000.00

Change any value to see how the forecast reacts. Picking a new store, department or week resets them.

ML Fundamentals Mini Project

Group 12

Polynomial Regression (degree 2)

Retaillence — Next-Week Sales Forecasting Engine

Forecast next week's sales for any department of any of 45 Walmart stores from its sales history, holidays and promotions, to plan stock and staff a week ahead.

Store 1 - Dept 1 - week of 2012-10-26

Forecast weekly sales

\$32,868

80% range: 29,680 - 36,056

Rule-of-thumb check

\$36,093

Actual sales that week

not known

The 16 weeks before, the same weeks last year, and this week's numbers

Weekly sales (\$)

Jul 08 - Oct 21

Actual sales Same week last year Model forecast Rule-of-thumb Actual (this week)

Store & department snapshot

Store format / size

Type A - 151,315 sq ft

Average week, past year

\$23,053

Last 4 weeks' average

\$25,904

Same week last year

\$31,580

Last year's jump for this week

+\$10,189

Kind of week

Normal week

Markdown (promotion) this week

\$50,000

Model performance (17 unseen test weeks)

Algorithm

Polynomial Regression (degree 2)

Trained on

2011-03-04 - 2013-06-29

Test R²

0.987

Test MAE / RMSE

\$1,277 / \$2,544

Error cut vs last week's sales

17.9%

Error cut vs rule-of-thumb

7.9%

Group attribution

Group 12 - ML Fundamentals

Inputs the model used for this forecast

Forecast inputs

Store

Store 16 - Type B - 57,197 sq ft

Department (product group)

Dept 83

Week to forecast (week ending Friday)

2011-09-02 - training week

ML Fundamentals Mini Project

Group 12

Polynomial Regression (degree 2)

Retaillence — Next-Week Sales Forecasting Engine

Forecast next week's sales for any department of any of 45 Walmart stores from its sales history, holidays and promotions, to plan stock and staff a week ahead.

Not enough history: Dept 83 in Store 16 has 12 weeks of sales before 2011-09-02, but the model needs 56 (it compares with the same week last year). Pick a later week or another department.

Testing with multiple inputs and input validation

21 / 21 tests pass

in 6.4 s (`pytest`).

Multiple inputs

5 store / department / week combinations: normal week, Labor Day, small Type C store, Thanksgiving, next-week forecast; each gives a forecast inside its range.

Validation

Negative markdowns (*price cuts / promotions*) are rejected; departments without 56 weeks of history get a clear error, not a number.

No leakage (*future information sneaking into training*)

A test proves a week's own sales never feed its inputs.

GROUP	TEST	
Streamlit app (multiple inputs + validation)	default view is a real next week forecast	✓
Streamlit app (multiple inputs + validation)	many inputs give a sane forecast and range[1-1-2012-10-26]	✓
Streamlit app (multiple inputs + validation)	many inputs give a sane forecast and range[20-92-2012-09-07]	✓
Streamlit app (multiple inputs + validation)	many inputs give a sane forecast and range[44-3-2012-07-06]	✓
Streamlit app (multiple inputs + validation)	many inputs give a sane forecast and range[35-72-2011-11-25]	✓
Streamlit app (multiple inputs + validation)	many inputs give a sane forecast and range[4-38-2012-11-02]	✓
Streamlit app (multiple inputs + validation)	holiday week is detected and warned about	✓
Streamlit app (multiple inputs + validation)	department without a year of history is rejected	✓
Streamlit app (multiple inputs + validation)	negative promotion is not accepted	✓
Streamlit app (multiple inputs + validation)	what if changes forecast and hides actual	✓
Streamlit app (multiple inputs + validation)	new week resets the what if inputs	✓
Feature pipeline	one row per week plus the forecast week	✓
Feature pipeline	lags look back the right number of weeks	✓
Feature pipeline	returns count as zero and missing weeks count as no sales	✓
Feature pipeline	promotion markdowns	✓
Feature pipeline	holiday features from the date	✓
Feature pipeline	only rows with 56 weeks of history are used	✓
Feature pipeline	no peeking at the week being predicted	✓
Saved model	saved model reproduces its reported test score	✓
Saved model	model beats both simple rules on the test weeks	✓
Saved model	model file matches installed scikit learn	✓

Results and discussion

QUESTION	ANSWER
Can next week's sales be predicted from history, store, product, promotion, month and holidays?	Yes: R^2 (<i>share of the ups and downs the model explains; 1 = perfect</i>) 0.987, average miss \$1,277 per department-week on unseen weeks.
Which algorithm is best?	Polynomial Regression (<i>linear regression plus squared and multiplied inputs, so the line can bend</i>) (degree 2), 17.9% better than last week's sales and 7.9% better than the seasonal rule on the test weeks.
Does polynomial beat linear?	Yes, modestly: CV (<i>cross-validation: testing on held-back blocks of weeks</i>) MAE (<i>average miss in dollars</i>) \$1,668 vs \$1,710, test \$1,277 vs \$1,309.
What drives sales?	Recent sales and the same week last year; holidays multiply them; department and month help a little.
Do promotions matter?	Not measurably once history is known (markdowns (<i>price cuts / promotions</i>) are anonymised and only reported from Nov 2011).
Is it good enough for inventory planning?	For normal weeks, yes. For big holiday weeks, check it against the rule of thumb; the app shows both.

CHECKLIST

Every requirement of the brief, answered

BRIEF (PROBLEM-STATEMENT.PDF)	WHERE IT IS DONE
Problem definition and clear objectives	Slides 5, 8 · notebook §1
Relevant dataset with source, feature description, target <i>(the number we predict)</i>	Slides 9–11 · notebook §2–3
Data cleaning and preprocessing	Slides 12–13 · notebook §4, §7–8
EDA <i>(exploring the data with charts)</i> with meaningful visualisations and interpretations	Slides 15–22 · notebook §6
Feature selection or feature engineering <i>(building new input columns from raw data)</i>	Slides 14, 25 · notebook §5, §10
Training and testing; appropriate metrics	Slides 23–26 · notebook §9–12
Model comparison and final model selection	Slides 26–27 · notebook §9–12
Saving/loading the trained model	<code>models/best_model.joblib</code> , reload check in notebook §15
Functional Streamlit <i>(Python tool for building a web app)</i> app with inputs and output	Slides 30–31 · <code>app.py</code>
Testing with multiple inputs and basic input validation	Slide 32 · <code>tests/</code>
Report, source code, dataset, model file, requirements.txt, README	This PDF · repo tree on slide 2
Individual contribution record and viva	Slide 37 · <code>CONTRIBUTIONS.md</code>

DISCUSSION

Limitations

One holiday season to learn from

"Same week last year" only exists from Feb 2011, and the test weeks contain no Thanksgiving or Christmas, so holiday accuracy is the least proven part.

One week ahead only

Forecasting further out would need the model's own forecasts as inputs.

Anonymised markdowns *(price cuts / promotions)*

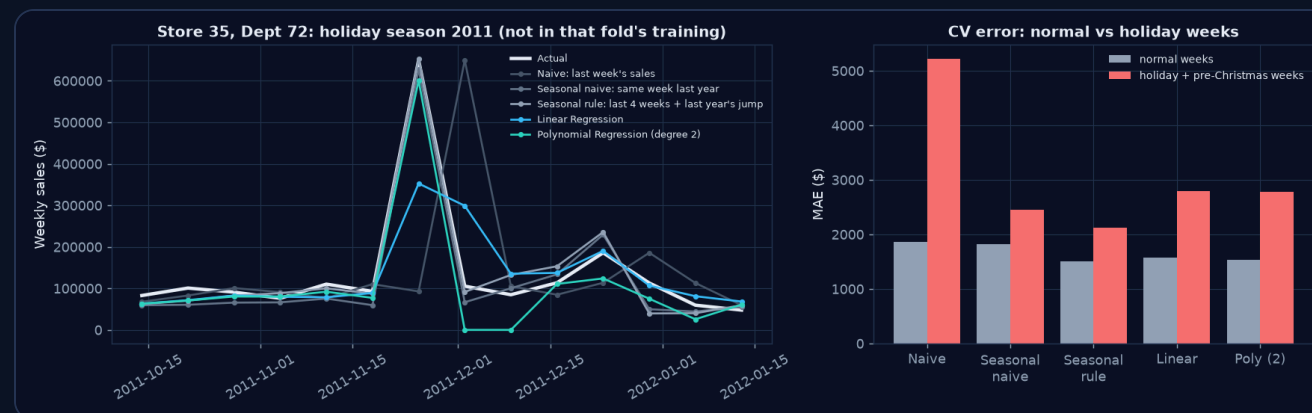
The true effect of promotions cannot be measured from this data.

Polynomial extrapolation *(predicting far outside what it has seen)*

Far outside the training range (e.g. an unseen \$650K spike) squared terms over-react; the app limits what-if inputs to the training range.

No outside events

Local festivals, weather shocks and competitor openings are not in the data.



CONCLUSION

Conclusion and future scope

Conclusion

A degree-2 Polynomial Regression (*linear regression plus squared and multiplied inputs, so the line can bend*) on engineered history, calendar and store features (*input columns the model uses*) forecasts next week's department sales with R^2 (*share of the ups and downs the model explains; 1 = perfect*) 0.987 and cuts the error of the best rule of thumb by 7.9%, deployed as a tested Streamlit (*Python tool for building a web app*) app.

Future scope

More years of data (more holiday seasons) · a dedicated holiday-week model · forecasts 2–4 weeks ahead · Ridge-regularised (*extreme weights are penalised to avoid over-fitting*) polynomial terms · live POS (*shop checkout data*) data feed.

\$1,277

average miss

0.987

R^2

7.9%

better than the rule

21/21

tests pass

Team contributions and references

MEMBER	ROLL NO.	OWNED
Anshuman Atrey	150096724029	see CONTRIBUTIONS.md
Shlok Kadam	150096724103	see CONTRIBUTIONS.md
Rajneesh Kumar	150096724144	see CONTRIBUTIONS.md

Full record in `CONTRIBUTIONS.md` . Every member explains one part in the viva.

1. Kaggle (*data-science competition website*) (2014). *Walmart Recruiting – Store Sales Forecasting*. kaggle.com/c/walmart-recruiting-store-sales-forecasting
2. A. Ahmedov (2022). *Walmart Sales Forecast* (dataset mirror). kaggle.com/datasets/aslanahmedov/walmart-sales-forecast
3. R. Fildes, S. Ma, S. Kolassa (2022). Retail forecasting: Research and practice. *International Journal of Forecasting* 38(4), 1283–1318.
4. S. Makridakis, E. Spiliotis, V. Assimakopoulos (2022). M5 accuracy competition: Results, findings, and conclusions. *IJF* 38(4), 1346–1364.
5. C. S. Bojer, J. P. Meldgaard (2021). Kaggle forecasting competitions: An overlooked learning opportunity. *IJF* 37(2), 587–603.
6. C. Bergmeir, J. M. Benítez (2012). On the use of cross-validation (*testing on several held-back blocks of weeks*) for time series predictor evaluation. *Information Sciences* 191, 192–213.
7. R. J. Hyndman, G. Athanasopoulos (2021). *Forecasting: Principles and Practice*, 3rd ed. OTexts.
8. F. Pedregosa et al. (2011). Scikit-learn: Machine learning in Python. *JMLR* 12, 2825–2830.
9. Streamlit (*Python tool for building a web app*) documentation. docs.streamlit.io